

Arduino: Ultraschall, IR und Displays

Worum geht es in diesem Blatt?

In diesem Blatt erweitern Sie den Arduino um Ausgabe und Abstandsmessung. Sie geben Text und Grafik auf einem OLED-Display aus, binden Infrarotsensoren an, probieren weitere Displaytypen aus und messen mit Ultraschall Entfernungen. Die letzte Aufgabe ist bewusst offen gehalten: Dort kombinieren Sie die Bausteine zu etwas Eigenem.

Was erwarten wir uns von der Praktikumsdokumentation/Protokoll?

- Beantworten der gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Zusätzlich machen Sie bitte Screenshots ihrer Programme sowie Fotos, die Ihre Lösungen in „Aktion“ zeigen; speichern der Programme nicht vergessen!
- Eigene Interpretation von Erkenntnissen des Experiments, Auffälligkeiten
- Welche Probleme sind Ihnen begegnet und wie haben Sie diese gelöst?

Wie?

- Beantworten Sie zur Abgabe den Fragebogen/Quiz „Moodle-Abgabe Blatt 4“ in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- Kopieren Sie dazu Ihre Antworten (Protokoll ggfs. in der Praktikumsstunde direkt im Angabe-Worddokument oder separatem Textdokument erstellen), Screenshots, Fotos, etc. in die jeweiligen Antwortfelder.
- **ACHTUNG: Chromium-basierte Browser scheinen keine Bilder kopieren zu können, nutzen Sie in dem Fall einen anderen Browser, z.B. Firefox**

Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Der große Vorteil der Arduino-Plattform ist, dass es für eine Vielzahl an Hardware-Komponenten bereits fertige Tutorials und/oder Programme gibt. Dadurch wird das Hinzufügen von weiteren Sensoren erheblich erleichtert. In diesem Aufgabenblatt werden wir an verschiedenen Beispielen praktisch ausprobieren, wie wir zusätzliche Sensoren finden und mithilfe von externen Bibliotheken in unseren Programmen einbinden können.

1) Meine ersten Ausgaben auf einem OLED-Display

Im ersten Schritt geht es darum, ein externes Display anzuschließen und Texte auf dem Display auszugeben

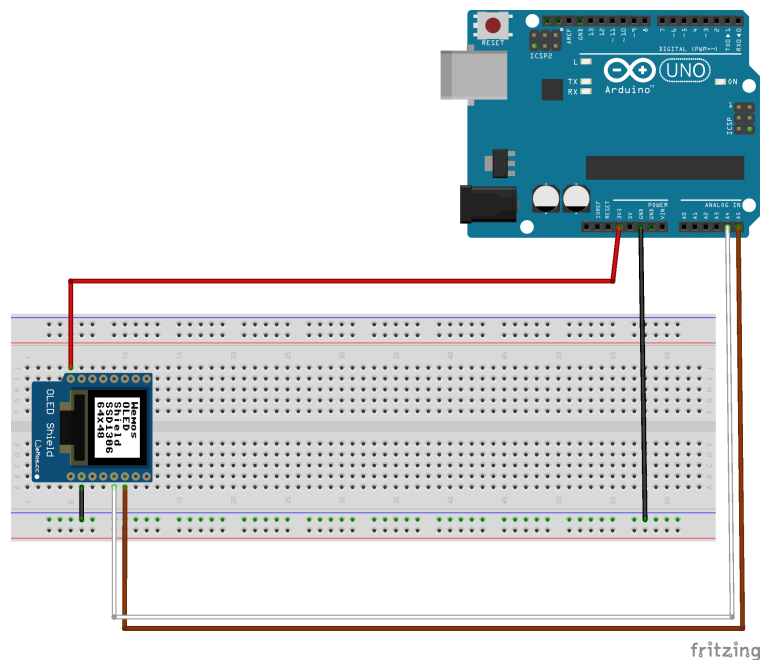
Wir benötigen folgende Komponenten

- Arduino UNO-Board
- „Breadboard“
- OLED Display
- Kabel

Vorbereitung

- Am PC die Arduino Entwicklungsumgebung starten
- Mittels USB-Kabel das Arduino-Board mit dem PC verbinden
- Die Arduino IDE wieder für den korrekten Chip konfigurieren (μ C auswählen): Menü „Tools“ → „Board“ → „Arduino UNO R4 WIFI“ auswählen
- Die serielle Schnittstelle konfigurieren: Menü „Tools“ → „Serial Port“ → „COM1“ (richtigen Port auswählen, falls nicht bekannt: siehe Windows-Systemsteuerung)
- Über die Option „File“ → „New“ einen neuen Sketch anlegen

Aufbau



- Schalten Sie den Arduino stromlos (USB-Kabel abstecken)

- Das Display **wird mit 3.3V betrieben**; verbinden Sie den linken unteren PIN des Displays (Beschriftung 3V3) mit dem 3V3 (oder VCC) Ausgang Ihres Arduino
- Verbinden Sie den zweiten PIN rechts unten des Displays (Beschriftung G) mit dem Ausgang GND des Arduinos. Es empfiehlt sich (in Vorbereitung auf die nächsten Aufgaben), den Ausgang des Arduino mit der Masse-Leiste des Breadboards zu verbinden und die Masse dann von dort abzugreifen (wie in der Abbildung dargestellt).
- Verbinden Sie den Ausgang D2 (SDA) des Displays mit dem Eingang A4 des Arduino
- Verbinden Sie den Ausgang D1 (SCL) des Displays mit dem Eingang A5 des Arduino

Nun können wir unsere erste Ausgabe auf dem Display anzeigen – bitte denken Sie auch hier daran, die beiden Adafruit Bibliotheken (Menü „Sketch“ → „Bibliothek einbinden“ → „Bibliotheken verwalten“ → „Adafruit GFX“ bzw. „Adafruit SSD1306“ suchen → „Install“) über die Arduino IDE zu installieren. **Wichtig:** Bei der Suche nach den Bibliotheken den Unterstrich der Pakete (z.B. in „Adafruit_GFX“) nicht mit eingeben.

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET 0
Adafruit_SSD1306 display(OLED_RESET);

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.display();

  display.setTextSize(1);
  display.setTextColor(WHITE);
}

void loop() {
  display.clearDisplay();
  display.setCursor(0, 0);

  display.println("TMI macht viel Spaß!");
  display.display();
  delay(50);
}
```

Frage 1: Funktioniert Ihr Programm? Wird der Text vollständig auf dem Display dargestellt? Wozu dient der Aufruf `display.setCursor(0, 0)`?

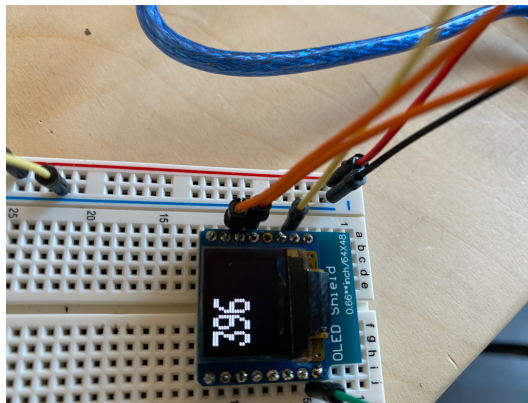
Der Treiber für einige unserer Displays ist ursprünglich für Displays der Größe 128 x 64 Pixel ausgelegt. Unser Display hat jedoch nur 64 x 48 Pixel. Das bedeutet, dass wir unter Umständen bei Ihrem Display nicht an der Position (0, 0) mit unserer Ausgabe beginnen dürfen.

Falls Sie nichts sehen: Passen Sie die Positionierung des Cursors daher nun so an, dass der Text in der linken oberen Ecke Ihres Displays beginnt.

Frage 2: Funktioniert Ihr Programm? Wird der Text vollständig auf dem Display dargestellt?

Bei Verwendung des Displays wird ganz offensichtlich die linke obere Ecke abgeschnitten – und auch der Zeilenumbruch nicht korrekt durchgeführt.

Passen Sie das Programm so an, dass der Text sauber auf mehrere Zeilen verteilt wird.

Frage 3: Wie muss der Code in „loop()“ aussehen, damit die Ausgabe vollständig auf dem Display zu sehen ist?

Wir wollen nun unser Programm so erweitern, dass dynamische Werte ausgegeben werden können:

- Wir initialisieren (außerhalb des Loops) eine Variable `i` vom Typ `integer` mit dem Wert `0`
- In unserer erweiterten `Setup`-Methode passen wir folgendes an: Wir wählen nun die Textgröße `2`.
- Innerhalb des Loops erfolgen der Reihe nach die folgenden Schritte:
- Wir löschen zunächst das Display
- Wir positionieren den Cursor an der richtigen Stelle
- Wir geben die variable `i` mithilfe der Methode `display.println` aus
- Wir zeigen den Text mit `display.display` an
- Wir warten `50 ms`.
- Wir erhöhen die Variable `i` um den Wert `1`

Frage 4: Was beobachten Sie? Kopieren Sie Ihren kompletten Sketch in die Antwort zu dieser Frage.

Frage 5: Funktioniert ihr Programm? Welchen Wert liefert der Sensor? Bewegen Sie Ihre Hand über den Sensor. Verändert sich der Wert? In welcher Form verändert sich der Wert?

Auf der Rückseite des Sensors befindet sich eine kleine Plastikschraube. Drehen Sie diese Schraube nun vorsichtig (!) mit einem passenden Schraubenzieher.

Frage 6: Welchen Effekt hat die Veränderung der Stellung dieser Schraube auf das Verhalten Ihres Programms?

Bringen Sie die Schraube nun wieder in eine mittlere Position, so dass der Sensor wie gewünscht arbeitet.

Passen Sie das Programm so an, dass der Wert des Sensors digital eingelesen wird.

Frage 7: Was wird nun auf dem Display ausgegeben? Welcher Code muss innerhalb der Loop-Methode stehen? Posten Sie den entsprechenden Ausschnitt aus Ihrem Sketch.

Wir wollen nun den IR-Sensor zu einem kontaktlosen Schalter erweitern. Das bedeutet: Jedes Mal, wenn die Hand über den Sensor fährt, dann soll dieser Kontakt einmal (!) gezählt werden. Solange die Hand über dem Sensor ist, werden keine weiteren Kontakte gezählt. Erst wenn die Hand vom Sensor entfernt wird (und dann wieder erneut über dem Sensor gehalten wird), wird ein weiterer Kontakt gezählt. Auf dem Display soll die Anzahl der kontaktlosen „Klicks“ angezeigt werden.

Gehen Sie dazu wie folgt vor:

- Wir wollen in unserem Programm eine sogenannte fallende Flanke (ein Wechsel von 1 auf 0) erkennen. Das bedeutet: Wir prüfen mithilfe einer Verzweigung, ob der aktuelle Sensorwert 0 und der Sensorwert des vorherigen Durchlaufs 1 ist.
- Gleichzeitig müssen wir in jedem Loop sowohl den aktuellen als auch den vorherigen Sensorwert kennen.
- Wir speichern diese Sensorwerte in den Variablen `previous` und `current` ab. Diese initialisieren wird jeweils (zu Beginn unseres Sketches; außerhalb des Loops) jeweils mit 1. Außerdem brauchen wir noch eine dritte Variable, nämlich `count`, um die Kontakte zu zählen. Diese initialisieren wir an der gleichen Stelle, jedoch mit dem Wert 0.
- Damit ergibt sich für den Loop folgende Grundstruktur

```
void loop() {  
  display.clearDisplay();  
  display.setCursor(32,8);  
  
  current = digitalRead(IR);  
  
  // Zähler hochzählen, wenn neuer Kontakt erkannt.  
  
  display.println(count);  
}
```

```
display.display();  
previous = current;  
}
```

- Ersetzen Sie nun den Kommentar mit dem entsprechenden Programmcode, der eine fallende Flanke erkennt und dann den Zähler um 1 erhöht.

Frage 8: Funktioniert Ihr Programm wie gewünscht? Durch welchen Code müssen Sie den Kommentar ersetzen, um die gewünschte Funktionsweise zu erreichen?

Öffnen Sie nun das Programm Autodesk Tinkercad (<https://www.tinkercad.com>) und entwickeln Sie einen Schaltplan für den Anschluss eines Displays (SSD1306) und zwei IR-Sensoren (FC-51). Die exakten Bauteile sind dort leider nicht verfügbar. Verwenden Sie stattdessen das Standard I2C-Display und einen IR-Sensor mit drei PINs. Alternativ können Sie das natürlich auch auf Papier zeichnen – aber mit Tinkercad sieht es einfach besser aus! Lassen Sie sich den Schaltplan von einem der Praktikumsbetreuer freigegeben, bevor Sie diesen auf dem Breadboard entsprechend umsetzen. Achten Sie darauf, dass Sie die Schaltung mit ausreichend Abstand zwischen den beiden IR-Sensoren aufbauen, so dass Sie unabhängig mit beiden Sensoren interagieren können.

Erweitern Sie nun Ihr Programm dahingehend, dass Sie zwei parallele Zähler haben, die jeweils die (weiterhin berührungslose) Kontakte unabhängig voneinander zählen. Starten Sie einen kleinen Wettbewerb mit *ihrer Teamkollegin*, wer den „eigenen“ Zähler schneller nach oben treiben kann.

Frage 9: Wie viele Variablen sind mindestens notwendig, damit die zwei Zähler fehlerfrei funktionieren?

3) Neue Displays – neue Möglichkeiten

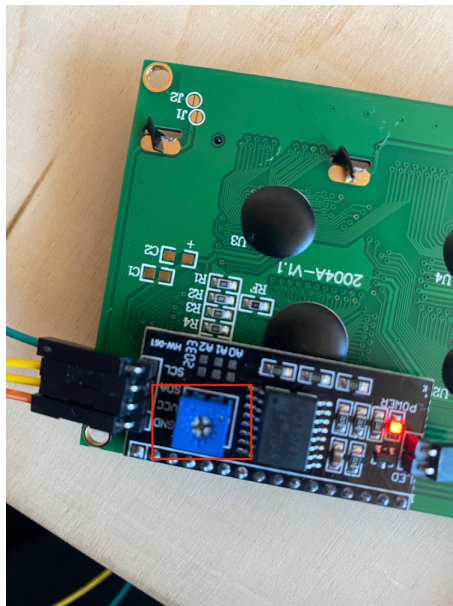
In Vorbereitung auf den Hackathon wollen wir nun lernen, wie wir beliebige Hardware mit unserem Arduino nutzen können, indem wir entsprechende Bibliotheken laden (und einbinden).

Ein besonders einfacher Weg ist dabei, die Hardware über Funduino (<https://funduino.de>) zu bestellen. Dort gibt es neben der Hardware auch Anleitungen zur Verwendung der entsprechenden Bauteile mit Ihrem Arduino.

Bei dem nun zu nutzenden Display handelt es sich um ein Vierzeiliges I2C LCD Display für Arduino (<https://funduino.de/anleitung-4x20-i2c-lcd-modul>). Führen Sie nun die folgenden Schritte aus der online verfügbaren Anleitung durch:

- Schließen Sie alle vier Leitungen des Displays am Arduino an; es empfiehlt sich, Masse und die 5V wieder über die Leisten am Breadboard zu beziehen. Achten Sie darauf, dass dieses **Display nun eine Spannung von 5V benötigt**.
- Prüfen Sie (wie erläutert) ob die passende Bibliothek auf Ihrem Rechner schon installiert ist. Falls nicht, dann laden Sie die Datei LiquidCrystal_I2C direkt aus Moodle herunter. Die Datei müssen Sie in Ihrem persönlichen Arduino Ordner (unter Benutzer / Dokumente) in dem Unterordner libraries entpacken.

Erstellen Sie einen neuen Sketch und kopieren Sie den Beispielcode in Ihren Sketch und übertragen Sie das Programm auf den Arduino



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,20,4);

void setup()
{
  lcd.init();
```

```

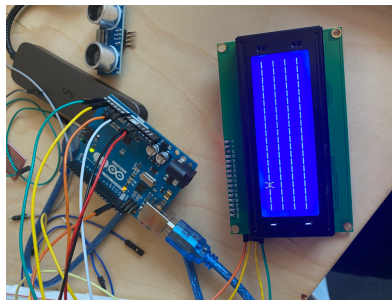
lcd.init();

lcd.backlight();
lcd.setCursor(3,0);
lcd.print("Hello World!");
lcd.setCursor(2,1);
lcd.print("Arduino UNO R4");
lcd.setCursor(0,2);
lcd.print("--");
lcd.setCursor(2,3);
lcd.print("TMI Praktikum");
}

void loop() {}

```

- Falls Sie keinen Text lesen können, dann ist das Display **vermutlich zu hell oder zu dunkel** eingestellt. Drehen Sie die Plastikschrube auf der Rückseite des Displays vorsichtig (!) bis Sie den Text problemlos lesen können.



Nun wollen wir eine kleine Animation auf unserem Display laufen lassen. Dazu definieren wir eine Variable `no` und eine Variable `yes` mit jeweils einem Zeichen unserer Wahl. Wenn wir ein `x` über – „fliegen“ lassen wollen, dann wählen wir als `yes` ein `x` und als `no` einen `-`.

Nun machen wir folgendes:

- Wir initialisieren die Variablen `yes` und `no` sowie einen Zähler (`count`); der Zähler startet bei 0. Außerdem brauchen wir eine Variable `text` vom Typ `String`, diese initialisieren wir mit einem leeren `String`.
- Die Methode `setup` bleibt unverändert.
- Innerhalb des `loops` füllen wir nun den `text` mithilfe einer `for`-schleife mit insgesamt 80 Zeichen (20 Zeichen pro Zeile, 4 Zeilen). Wenn der Schleifenzähler dem Wert `count` entspricht, dann hängen wir das Zeichen `yes` an – in allen anderen 79 Fällen wird das Zeichen `no` angehängt.
- Anschließend wird der so gefüllte `text` ausgegeben; dazu positionieren wir den Cursor in der linken oberen Ecke und nutzen dann die `println`-Methode des `lcd`-Displays.
- Zum Schluss wird der Counter noch um eins erhöht
- Wenn der Counter über 80 ist, beginnt das Ganze wieder von vorne (dazu können Sie beispielsweise den Modulo-Operator, das `%`-Zeichen verwenden)
- Bauen Sie am Ende des `Loops` noch ein `delay` von 10 ms ein.

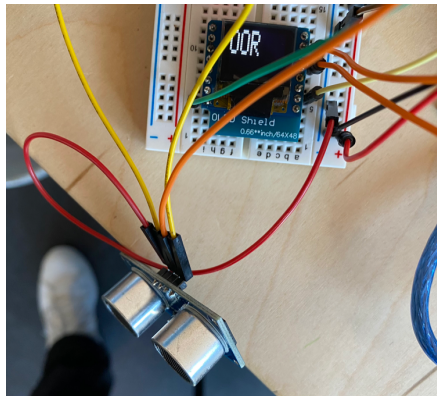
Frage 10: Wie „fliegt“ das X nun über den Bildschirm? Hätten Sie dieses Verhalten erwartet? Woran könnte das liegen? Wie lässt sich das Problem beheben? Sie müssen an dieser Stelle das Problem nicht lösen, sondern es genügt, die Lösungsidee zu skizzieren.

Nun sind Sie perfekt vorbereitet, um im Rahmen des Hackathons schnell mit neuen Sensoren arbeiten zu können!

4) Ultraschall

Wir haben ja in Aufgabe 2 bemerkt, dass der IR-Sensor nicht wie erhofft direkt für die Messung von Entfernungen verwendet werden kann. Um die Position Ihrer Hand halbwegs präzise erraten zu können, bräuchten wir ein ganzes Array von IR-Sensoren.

Doch zum Glück gibt es ja noch Ultraschallsensoren, die deutlich besser für die Messung von Entfernungen geeignet sind.



Wir benötigen daher nun zusätzlich folgende Komponente

- Ultraschallsensor HC-SR04

Es steht Ihnen frei, ob Sie das erste oder das zweite Display für diese Aufgabe nutzen möchten. In dem nachfolgenden Schaltplan ist die Variante mit dem ersten Display zu sehen. Hierbei muss man besonders gut darauf achten, das **Display mit 3V3 und den Ultraschallsensor mit 5V** zu versorgen.

Unabhängig davon, welches Display Sie verwenden, wird der Ultraschallsensor wie folgt angeschlossen:

- GND wird mit der Masseleiste und VCC mit der 5V-Leiste am Breadboard verbunden
- Der Trigger (Trig) wird mit einem digitalen Port, z.B. Port 7 verbunden
- Das Echo wird ebenfalls mit einem digitalen Port, z.B. Port 6 verbunden

Eine einfache Ansteuerung des Sensors sieht dann wie folgt aus:

```
// je nach Display entsprechende Imports

#define PIN_TRIGGER 7
#define PIN_ECHO    6
#define OLED_RESET 0
unsigned long duration;

void setup() {
    pinMode(PIN_TRIGGER, OUTPUT);
    pinMode(PIN_ECHO, INPUT);

    // Initialisierung des Displays
}
```

```

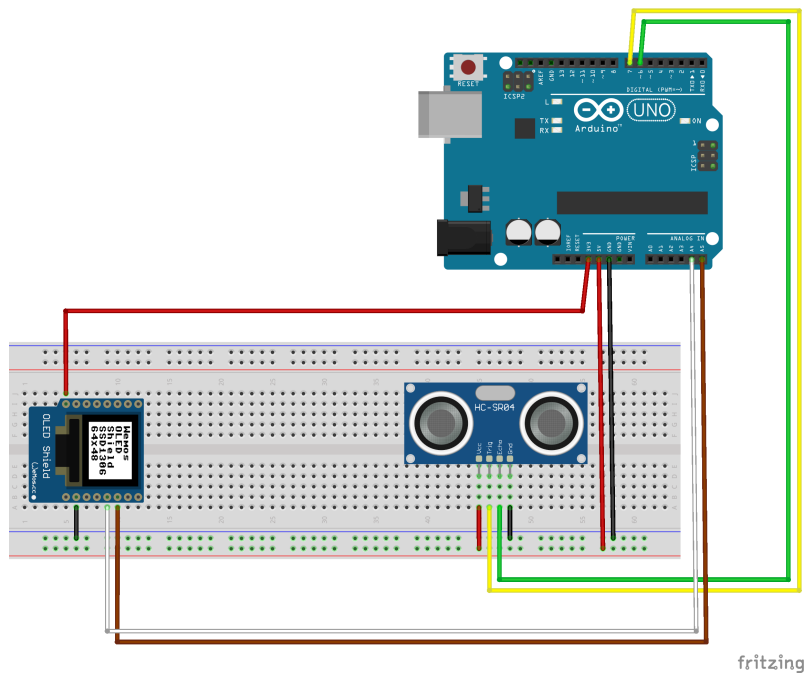
void loop() {
  // Display leeren

  digitalWrite(PIN_TRIGGER, LOW);
  delay(5);
  digitalWrite(PIN_TRIGGER, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIGGER, LOW);

  duration = pulseIn(PIN_ECHO, HIGH);

  // Variable duration auf dem Display ausgeben

  delay(100);
}
  
```



Versuchen Sie nun das Programm nachzuvollziehen:

- Wir nutzen den Trigger als Output und das Echo als Input
- Wir legen nun am Trigger kurz (d.h. 10 μ s) einen HIGH-Pegel an; dadurch sendet der Sensor ein Schallsignal
- Dann schalten wir das Signal wieder aus und lesen an dem Pin Echo mit `pulseIn` die Laufzeit aus

Frage 11: Bewegen Sie sich nun langsam vom Sensor weg. Wie verändert sich die Laufzeit? In welchem Bereich kann der Sensor (ungefähr) hinreichend große Objekte (z.B. eine Wand) detektieren?

Nun ist es ja nicht ganz so spannend, zu erfahren, wie viele Mikrosekunden der Schall zum Objekt braucht. Viel hilfreicher wäre es doch, wenn wir die Entfernung erfahren könnten! Kein Problem, die Schallgeschwindigkeit ist ja erfreulicherweise bekannt:

- Die Schallgeschwindigkeit beträgt 1235,5 km/h (in trockener Luft)
- Das entspricht 343,2 m/s
- Das sind 34.320 cm/s – doch die Messung von `pulseIn` erfolgt in μs (eine Millionstel Sekunde, 0,000001 Sekunden).
- Wenn wir nun also unseren Messwert (in μs) mit der Schallgeschwindigkeit (in cm/ μs) multiplizieren, dann erhalten wir die Entfernung des Objekts in cm.

Passen Sie den Code entsprechend an – und geben Sie die Entfernung aus. Testen Sie empirisch, ob Ihre Berechnung richtig ist. Vermutlich haben Sie sich nun zu einem kleinen Denkfehler verleiten lassen. Denken Sie noch einmal über die Funktionsweise des Sensors nach und korrigieren Sie Ihre Berechnung.

Frage 12: Wie lautet die korrekte Formel für die Berechnung der Distanz in cm?

Wir wollen nun die Distanz nur ausgeben, wenn diese zwischen 0 und der maximalen Reichweite liegt. Wenn der Messwert sich hingegen außerhalb dieses Bereichs befindet, dann geben wir stattdessen 00R (für „out of range“ aus).

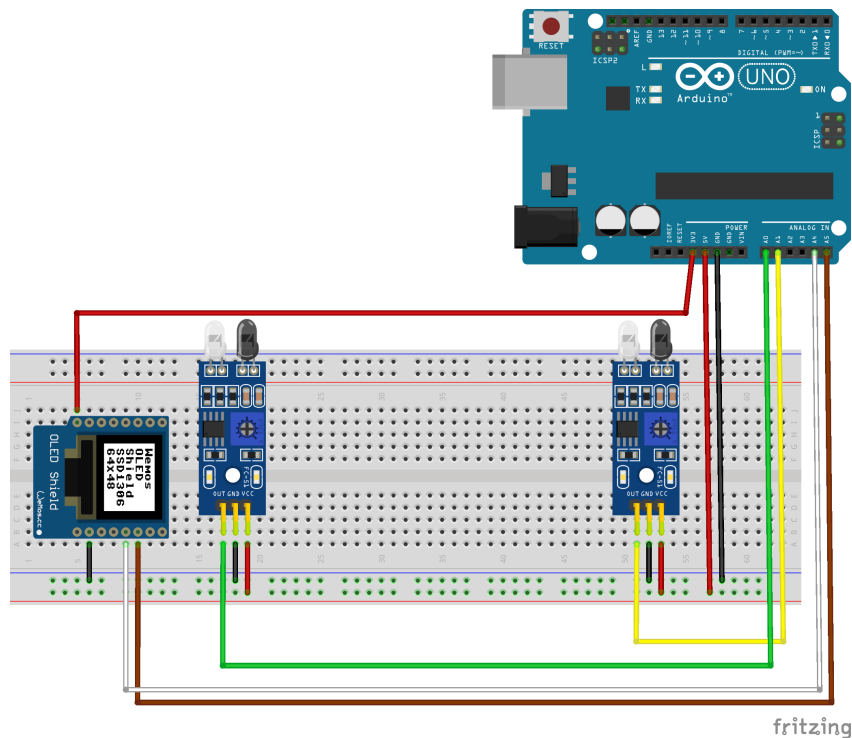
Frage 13: Wie sieht die finale Version des Codes innerhalb der Loop-Methode aus?

5) Kreativ werden

Falls Sie nun noch Zeit haben, können Sie die bisherigen Sensoren für eine kleine, kreative Anwendung nutzen. Beispielsweise können Sie die beiden IR-Sensoren nur als kontaktlose Buttons für ein kleines Quiz verwenden.

Aber grundsätzlich können Sie Ihrer Kreativität freien Lauf lassen, Sie können statt eines Quizzes nun beispielsweise auch auf dem Beispielprogramm zu dem SSD1306-Display aufsetzen (zu finden unter „Datei -> Beispiele -> Beispiele aus eigenen Bibliotheken -> Adafruit SSD 1306 -> ssd1306_128x64_i2c“) und die Animation ggf. an das kleinere Display anpassen, indem Sie wieder den Offset von 32, 8 entsprechend berücksichtigen.

Wenn Sie jedoch zum Abschluss das Quiz bauen möchten, dann gehen Sie bitte wie folgt vor: Verkabeln Sie eines der beiden Displays und die beiden IR-Sensoren:



Anschließend öffnen Sie das auf Moodle verfügbare Beispiel (IR_Kollisionen_quiz.ino). In dem Quiz wird für die Speicherung der Fragen ein zweidimensionales Array verwendet:

```
String fragen[3][2] = { { "Ist heute", "Montag?" }, { "Ist heute", "Dienstag?" }, { "Ist heute", "Mittwoch?" } };
```

Jede Frage besteht dabei in diesem Beispiel aus zwei Zeilen (falls Ihre Fragen länger sind, lässt sich das aber problemlos anpassen) und insgesamt gibt es drei Fragen. Fügen Sie nun im ersten Schritt insgesamt 10 Fragen ein. Denken Sie daran, die erste Dimension des Arrays auf 10 zu erhöhen. Falls Sie mehr als zwei Zeilen benötigen, passen Sie die zweite Dimension ebenfalls entsprechend an.

Die Antworten werden wie folgt codiert:

```
int antworten[3] = { 0, 0, 1 };
```

Die Länge des Arrays Antworten entspricht dabei natürlich der ersten Dimension des Fragen-Arrays, in diesem Beispiel also zunächst drei. Ist eine Frage falsch, ist die Antwort 0 – ist die Frage hingegen wahr, so lautet die Antwort 1.

Die Methode punktestand dient dazu, um nach jeder Beantwortung den Zwischenpunktestand auszugeben. Ist das Quiz vorbei, wird damit der Endpunktestand ausgegeben.

In einem Spiel werden wie folgt die Fragen der runde ausgegeben:

```
display.setCursor(32,8);
display.println(fragen[runde][0]);
display.setCursor(32,16);
display.println(fragen[runde][1]);
display.setCursor(32,24);
display.println("ja / nein");
```

Falls Ihre Fragen aus drei Zeilen bestehen, dann ersetzen Sie die letzte Zeile („ja / nein“) durch die Ausgabe Ihrer dritten Zeile.

Wenn richtige Fragen mit ja und falsche Fragen mit nein beantwortet werden, dann gibt es einen Punkt. Je nach Verkabelung müssen Sie die Logik ggf. invertieren, damit der Sensor für ja auf der linken und der Sensor für nein auf der rechten Seite ist:

```
if ((prev_ir1 == 1) && (ir1 == 0)) {
  // ja
  if (antworten[runde] == 1) {
    punkte++;
  }
  (...)
} else if ((prev_ir2 == 1) && (ir2 == 0)) {
  // nein
  if (antworten[runde] == 0) {
    punkte++;
  }
  (...)
}
```

Da Sie mehr als drei Fragen haben, müssen Sie nun noch den Spielabbruch entsprechend anpassen.

Frage 14: Posten Sie als Antwort auf die letzte Frage bitte den Inhalt Ihres „kreativen“ Arduino Sketches.